

You are receiving this e-mail because you elected to receive e-mail from Sun Microsystems, Inc. To update your communications preferences, please see the link at the bottom of this message. We respect your privacy and post our privacy policy prominently on our Web site <http://sun.com/privacy/>

Please do not reply to the mailed version of the newsletter, this alias is not monitored. Feedback options are listed in the footer for both content and delivery issues.



Core Java Technologies TECHNICAL TIPS

Welcome to the Core Java
Technologies Tech Tips.

January 13, 2006
[View this issue as simple text](#)

Welcome to the Core Java Technologies Tech Tips for January 13, 2006. Here you'll get tips on using core Java technologies and APIs, such as those in Java 2 Platform, Standard Edition (J2SE).

This issue covers:

- » The Singleton Pattern
- » The Observer Pattern

These tips were developed using Java 2 Platform, Standard Edition Development Kit 5.0 (JDK 5.0). You can download JDK 5.0 at <http://java.sun.com/j2se/1.5.0/download.jsp>.

This issue of the Core Java Technologies Tech Tips is written by John Zukowski, president of JZ Ventures, Inc. (<http://www.jzventures.com/>).

See the [Subscribe/Unsubscribe](#) note at the end of this newsletter to subscribe to Tech Tips that focus on technologies and products in other Java platforms.

A design pattern is a general solution to a common problem in software design. The idea is that the solution gets translated into code, and that the code can be applied in different situations where the problem occurs. Discussion of design patterns started with the book [*Design Patterns: Elements of Reusable Object-Oriented Software*](#) by Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. These four authors are known as the "Gang of Four" or simply "GoF". In their book, the GoF categorized patterns into different subject areas, of which there are three primary areas: creational, structural, and behavioral. Creational patterns describe how

objects are created (or "instantiated" in object-oriented terminology). Structural patterns offer help in how to connect and combine objects. Behavioral patterns describe algorithms or communication mechanisms. Some common pattern names are Singleton in the creational area, Observer in the behavioral area, and Facade in the structural area. This Tech Tip describes the Singleton pattern strictly in the context of the Java programming language.

One of the commonly used creational patterns is the Singleton pattern. It describes a technique for ensuring that only a single instance of a class is ever created. In essence, the technique takes the following approach: don't let anyone outside the class create instances of the object. Typically, Singletons are lazily created to reduce memory requirements until needed. You can implement this approach in many different ways.

If you know the one instance being created will be a subclass, make the parent class abstract and provide a method to get the current instance. An example of this is the `Toolkit` class in the AWT package. The constructor for `Toolkit` is public (the default constructor in this particular case):

```
public Toolkit()
```

and the class has a `getDefaultToolkit()` method for getting the specific subclass -- in this case, the subclass is platform-specific:

```
public static Toolkit getDefaultToolkit()
```

On a Linux platform with the Sun Java runtime, the specific subclass is of type `sun.awt.X11.XToolkit`. However you don't need to know that because you only access the class through its common abstract parent class, `Toolkit`.

The `Collator` class is another example of this pattern, with a slight difference. It offers two `getInstance()` methods. The no-argument version gets the `Collator` for the default locale. You can pass in your own locale to get the instance of the `Collator` for that locale. Request the `Collator` for the same locale multiple times and you get back the same `Collator` instance. The constructor itself is protected. Similar ways of restricting class creation can be found throughout the J2SE standard libraries.

At this point you might think that restricting access to the constructor of a class automatically makes it a Singleton. It doesn't. A case in point is the `Calendar` class. The `Calendar` class constructor is protected, and the class offers a `getInstance()` method to get an instance of the class. However, each call to `getInstance()` gets a new instance of the class. So that isn't a Singleton.

When you create your own Singleton class, make sure that only a single instance is ever created:

```
public class MySingleton {
    private static final MySingleton INSTANCE =
        new MySingleton();

    private MySingleton() {
    }

    public static final MySingleton getInstance() {
        return INSTANCE;
    }
}
```

The static method, `getInstance()`, returns the single instance of the class. Note that even if the single instance needs to be a subclass, you don't have to change the API.

Theoretically, you don't need the `getInstance()` method because the `INSTANCE` variable could be public. However, the `getInstance()` method does provide flexibility in case of future system changes. Good virtual machine implementations should inline the call to the static

`getInstance()` method.

That's not quite all there is to creating a Singleton. If you need to make your Singleton class `Serializable`, you must provide a `readResolve()` method:

```
/**
 * Ensure Singleton class
 */
private Object readResolve() throws ObjectStreamException {
    return INSTANCE;
}
```

With the `readResolve()` method in place, deserialization results in the one (and only one) object -- the same object as produced by calls to the `getInstance()` method. If you don't provide a `readResolve()` method, an instance of the object is created each time you deserialize the object.

The Singleton pattern is useful if you know you only have a single resource, and need to share access to the state information of that single resource. Identifying the need for the Singleton pattern at design time can simplify development. However, sometimes you're not aware of the need until a performance problem leads you to refactor code and use the pattern at a later time. For example, you might discover that system performance is degrading because your program is repeatedly creating instances of the same class to pass along state information. By changing to the Singleton pattern, you avoid recreating the same object. This frees up time the system uses to recreate the object, and saves the time the garbage collector needs to free those instances.

In short, use the Singleton design pattern when you want to ensure that one, and only one, instance of a class is ever created. If your constructor doesn't require any operations, provide an empty private constructor (or a protected constructor if you need to subclass). Otherwise, by default, the system will provide a public constructor, something you don't want when working with a Singleton.

Note that Singletons are only guaranteed to be unique within a given class loader. If you use the same class across multiple distinct enterprise containers, you'll get one instance for each container.

A Singleton pattern is often used with another pattern called the Factory pattern. Like the Singleton pattern, the Factory pattern is a creational pattern. It describes how subclasses of a particular object, or more typically, implementers of a particular interface, do the actual object creation. A good example of the Factory pattern is the Swing `BorderFactory` class. The class has a series of static methods returning different types of `Border` objects. It hides the implementation details of the subclasses, allowing the factory to directly call the constructors for the interface implementations. Here's an example of `BorderFactory` in use:

```
Border line = BorderFactory.createLineBorder(Color.RED);
JLabel label = new JLabel("Red Line");
label.setBorder(line);
```

Here, the fact that `BorderFactory` creates a `LineBorder`, or how `BorderFactory` does that, is hidden from the developer. In this particular example, you can directly call the `LineBorder` constructor, but in many cases of using the Factory pattern, you can't.

Frequently, the class implementing the Singleton pattern returns an object to use as a Factory to create instances of a different class. This is exemplified by the `PopupFactory` class in the way it creates `Popup` objects.

To get the Singleton factory, you call the `getSharedInstance()` method of `PopupFactory`:

```
PopupFactory factory = PopupFactory.getSharedInstance();
```

Then you create a `Popup` object from the factory by calling the factory's `getPopup()` method, passing in the parent component, its contents, and position:

```
Popup popup = factory.getPopup(owner, contents, x, y);
```

You'll find the Factory pattern used frequently in a security context. In the following example, a certificate factory is obtained for a particular algorithm, then a certificate for a stream is generated:

```
FileInputStream fis = new FileInputStream(filename);
CertificateFactory cf =
    CertificateFactory.getInstance("X.509");
Collection c = cf.generateCertificates(fis);
```

As shown with `BorderFactory`, the Factory pattern does not have to be used with the Singleton pattern. However the two patterns are frequently used together.

[Back to Top](#)

Like the Singleton Pattern, which was covered in the first half of this tip, the Observer pattern is a popular design pattern used in Java programs. The pattern is a behavioral design pattern. It defines a way for classes to be loosely coupled and for one class (or many) to be notified when another is updated. Basically, this means that when something happens in one place, you notify anyone who is observing and that is interested in that one place.

There are two ways to look at the Observer pattern. The first way involves the `Observer` and `Observable` classes found in the `java.util` package. The second way follows the JavaBeans component model of registering event listeners with components.

Prior to the creation of the JavaBeans event model, the `Observer` and `Observable` classes described an implementation of the Observable pattern. In other words, the classes have been around since the 1.0 version of the Java platform. There is nothing technically wrong with the classes and they are still present in the libraries. The classes still could be used to implement the Observable pattern, but the second model, the JavaBeans component model, is typically used. One significant problem in using the classes to implement the Observable pattern is that you have to extend `Observable`. This forces a class hierarchy structure that might not be possible in the single-inheritance world of the Java platform.

The JavaBeans component model of registering event listeners involves a series of add and remove methods, where the listener type is embedded in the method name. For instance, to observe the selection of a button, you register an `ActionListener` with the component:

```
ActionListener listener = new ActionListener() {
    public void actionPerformed(ActionEvent actionEvent) {
        ...
    }
};
JButton button = new JButton("Pick Me");
button.addActionListener(listener);
```

That really is all there is to the Observer pattern for the system-defined classes. You implement a listener interface, attach it to the Subject of the observation, and wait. The Subject is what is observed. It is responsible for remembering who is observing. In the JavaBeans component model case, the interface to attach and detach the `Observer` objects is the add/remove listener naming pattern. When the state of the Subject changes, it notifies the `Observer` objects.

One of the main objectives of the pattern is to enable the loose coupling of the Subject and

Observer. When the `JButton` is selected, instead of calling a specific method of a fictitious subclass named `ButtonNotification`, the notification is abstracted out into an interface that anyone can implement. The `JButton` doesn't care what class the attached `Observer` (listener) is. In fact, the button doesn't care if the implementing class is modified. All it cares about is that the `Observer` implements a listener.

There are a number of complications you need to watch out for when using the Observer pattern. First is the possibility of a memory leak. A reference to the `Observer` is maintained by the `Subject`. Until the `Subject` releases the reference, the `Observer` cannot be removed by the garbage collector. Be aware of this possibility and remove observers where appropriate. Also note that the set of `Observer` objects is maintained in an unordered collection -- at least when registering event listeners. You don't necessarily know if the first registered listener is notified first or last. If you need to have cascading notifications, where object A must be notified first, followed by object B, you must introduce an intermediary object to enforce the ordering. Simply registering the observers in a particular order will not enforce their order of notification.

Another area of the Java platform that models the Observer pattern is the Java Message Service (JMS), with its guaranteed delivery, non-local distribution, and persistence, to name a few of its benefits. The JMS publish-subscribe messaging model allows any number of subscribers to listen to topics of interest. When a message for the published topic is produced, all the associated subscribers are notified.

There are many other places in the Java platform that model the Observer pattern -- the pattern is frequently used throughout the Java platform.

Since the 1995 publication of the *Design Patterns* book by the Gang of Four, other books have been published that offer different perspectives on the same patterns and introduce additional patterns. Two of the more popular titles are:

- *Head First Design Patterns*
- *Refactoring to Patterns*

Other books, such as *Patterns of Enterprise Application Architecture* are available for those interested in more targeted design pattern coverage -- in this case, for enterprise applications.

For more information on design patterns in general, see its [Wikipedia entry](#).

This Tech Tips issue is dedicated to the memory of John Vlissides, one of the original Gang of Four authors. He passed away November 2005.

[Back to Top](#)

Rate and Review

Tell us what you think of the content of this page.

Excellent Good Fair Poor

Comments:

If you would like a reply to your comment, please submit your email address:

Note: We may not respond to all submitted comments.

Comments? Send your feedback on the Tech Tips:

<http://developers.sun.com/contact/feedback.jsp?category=newslet>

Subscribe to the following newsletters for the latest information about technologies and products in other Java platforms:

- Enterprise Java Technologies Tech Tips. Get tips on using enterprise Java technologies and APIs, such as those in the Java 2 Platform, Enterprise Edition (J2EE).
- Wireless Developer Tech Tips. Get tips on using wireless Java technologies and APIs, such as those in the Java 2 Platform, Micro Edition (J2ME).

You can subscribe to these and other Java technology developer newsletters or manage your current newsletter subscriptions on the [Sun Developer Network Subscriptions page](#)

ARCHIVES: You'll find the Core Java Technologies Tech Tips archives at:

<http://java.sun.com/developer/JDCTechTips/index.html>

IMPORTANT: Please read our Terms of Use, Privacy, and Licensing policies:

<http://www.sun.com/share/text/termsofuse.html>

<http://www.sun.com/privacy/>

http://developer.java.sun.com/berkeley_license.html

© 2006 Sun Microsystems, Inc. All Rights Reserved. For information on Sun's trademarks see:

<http://sun.com/suntrademarks>

Java, J2EE, J2SE, J2ME, and all Java-based marks are trademarks or registered trademarks of Sun Microsystems, Inc. in the United States and other countries.

Sun Microsystems, Inc. 10 Network Circle, MPK10-209 Menlo Park, CA 94025 US